18.2 How agents interact with external systems

As Trudy and the engineering team examined ACME's early agent deployments, another architectural pattern quickly became evident. Agents rarely work alone. To perform useful tasks, they need to communicate with other systems.

In most agent frameworks, these interactions happen through some type of tool invocation. A tool might retrieve documents, query a database, call an internal API, or trigger an automated workflow. The agent decides which tools to use as it works toward a goal.

Different systems present these capabilities in various ways. Some use structured tool interfaces where available operations are clearly described and accessed through defined schemas. Others enable agents to interact with systems via command-line interfaces or scripted environments. In both cases, the fundamental pattern remains the same: the agent requests to perform some operation outside the model itself.

From an authorization perspective, this pattern is essential. Every time an agent invokes a tool or executes a command, it is trying to perform an action on a resource. Conceptually, these requests follow the same *PARC model* introduced earlier in the book. Each request specifies the *principal* making the request, the *action* it plans to take, the affected *resource*, and the *context* in which it occurs.

The key difference is in how often they occur. Traditional applications usually follow predictable workflows, leading to a few authorization checks during a transaction. Agents, on the other hand, might try many actions while testing different strategies to solve a problem.

This means authorization decisions are no longer occasional checkpoints in a workflow. They become a continuous part of the interaction between the agent and the surrounding systems.

Figure 18.1 illustrates this relationship. An agent chooses a tool or command to invoke, submits a request describing the intended operation, and the policy engine evaluates the request before executing the action.